AGENTE DE TRAZAS OLLAMA

Arquitectura Completa y Guía de Usuario

Versión: 1.0.0
Fecha: Septiembre 2026
Estado: Producción
Última Actualización: 2026-09-14 12:05

■■ AGENTE DE TRAZAS OLLAMA

Arquitectura Completa y Guía de Usuario

• Versión**: 1.0.0

• Fecha**: Septiembre 2026

• Estado**: Producción

• Última Actualización**: 2026-09-14

■ TABLA DE CONTENIDOS

1. [Resumen Ejecutivo](#resumen-ejecutivo)

2. [Visión General de la Solución](#visión-general-de-la-solución)

3. [Arquitectura del Sistema](#arquitectura-del-sistema)

4. [Componentes Principales](#componentes-principales)

5. [Flujo de Datos](#flujo-de-datos)

6. [Requisitos Técnicos](#requisitos-técnicos)

7. [Instalación y Configuración](#instalación-y-configuración)

8. [Guía de Uso](#guía-de-uso)

9. [API REST - Documentación](#api-rest-documentación)

10. [Casos de Uso](#casos-de-uso)

11. [Optimización de Rendimiento](#optimización-de-rendimiento)

12. [Troubleshooting](#troubleshooting)

13. [Roadmap Futuro](#roadmap-futuro)

■ RESUMEN EJECUTIVO

¿Qué es el Agente de Trazas OLLAMA?

El **Agente de Análisis de Logs con OLLAMA** es una solución integral que integra:

• OLLAMA**: Motor LLM (Large Language Model) local para procesamiento de lenguaje

• Python + Flask**: API REST para orquestación de análisis

• Mulesoft Omni Gateway**: Gateway empresarial que expone la solución

• ServiceNow Integration**: Creación automática de incidentes

Capacidades Principales

■ **Análisis Inteligente de Logs** - Detección automática de patrones y anomalías

■ **Diagnóstico Asistido por IA** - Generación de reportes con recomendaciones

■ **Correlación de Eventos** - Identificación de causas raíz

■ **Integración ServiceNow** - Creación automática de tickets

■ **API REST Escalable** - Acceso desde cualquier plataforma

■ **Análisis Local** - Sin enviar datos a la nube

■ **Alto Rendimiento** - Latencia optimizada (< 30 segundos)

Beneficios

■ VISIÓN GENERAL DE LA SOLUCIÓN

Stack Tecnológico

```
■■■
```

Componentes de Alto Nivel

1. **ENTRADA: Mulesoft Omni Gateway**

• Responsabilidad**: Orquestar requests HTTP desde clientes

• Puertos**: 8081 (REST API)

• Tecnología**: Mulesoft ESB (Enterprise Service Bus)

• Características**:

- Validación de requests

- Transformación de datos (DataWeave)

- Manejo de errores

- Logging centralizado

- Rate limiting (configurable)

2. **ORQUESTACIÓN: Log Analyzer Agent (Python)**

• Responsabilidad**: Coordinación central de análisis

• Puertos**: 5000 (Flask API)

• Tecnología**: Python 3.9+, Flask

• Características**:

- Caching de respuestas

- Gestión de skills

- Parsing robusto de logs

- Manejo de errores

- Logging detallado

3. **PROCESAMIENTO: Skills Module**

• Responsabilidad**: Lógica especializada de análisis

• 7 Skills Disponibles**:

1. **log_ingestion.py** - Extracción y validación de logs

2. **log_pattern_matcher.py** - Identificación de patrones

3. **log_correlation_engine.py** - Correlación de eventos

4. **anomaly_detector.py** - Detección de anomalías

5. **log_report_generator.py** - Generación de reportes

6. **servicenow_token_generator.py** - Autenticación ServiceNow

7. **servicenow_incident_creator.py** - Creación de tickets

4. **IA: OLLAMA LLM**

• Responsabilidad**: Análisis semántico y generación de diagnósticos

• Puertos**: 11434

• Modelos Soportados**:

- llama2 (recomendado)

- phi (ligero, mejor latencia)

- qwen (alternativa)

• Características**:

- Inferencia local

- Sin dependencia de APIs externas

- Control total sobre datos

5. **INTEGRACIÓN: ServiceNow**

• Responsabilidad**: Gestión de incidentes y tickets

• Tipo**: REST API Cloud

• Características**:

- Autenticación OAuth 2.0

- Creación automática de tickets

- Vinculación con CMDB

- Histórico de análisis

■ COMPONENTES PRINCIPALES

A. COMPONENTE: MULESOFT OMNI GATEWAY

• Ubicación**: `log-analyzer-omni-gateway/`

• Estructura**:

```
log-analyzer-omni-gateway/ ■■■ pom.xml # Configuración Maven ■■■ mule-artifact.json # Metadatos Mule ■■■ src/main/mule/ ■■■ log-analyzer-omni-gateway.xml # Flows principales ■■■ global-configs.xml # Configuraciones globales
```

• Flows Disponibles**:

1. **health-check-flow**

- Endpoint: `GET /api/health`

- Validación de servicios disponibles

- Tiempo Promedio: < 100ms

2. **analyze-logs-flow**

- Endpoint: `POST /api/analyze`

- Análisis completo con IA

- Tiempo Promedio: 15-30 segundos

3. **analyze-patterns-flow**

- Endpoint: `GET /api/patterns`

- Análisis rápido sin IA

- Tiempo Promedio: < 2 segundos

4. **create-incidents-flow**

- Endpoint: `POST /api/incidents`

- Creación de tickets en ServiceNow

- Tiempo Promedio: 2-5 segundos

• Configuración Global**:

• URL del Agent: `http://localhost:5000`

• Timeout: 30 segundos

• Retry Policy: 3 intentos

• Content-Type: `application/json`

• Max Payload: 100MB

B. COMPONENTE: PYTHON AGENT

• Ubicación**: `log-analyzer-agent/`

• Archivos Principales**:

1. **api.py** - API REST Flask

- Define todos los endpoints

- Manejo de errores robusto

- Validación de input/output

- CORS habilitado

- Max request size: 100MB

2. **agent.py** - Orquestador Principal

- Coordina skills

- Gestiona caché

- Streaming de respuestas

- Limpieza de texto

- Métricas de latencia

3. **agent_optimized.py** - Versión optimizada

- Cache hash-based

- Streaming mejorado

- Mejor manejo de timeout

- Logging de latencia

4. **interactive_chat.py** - Chat interactivo

- CLI para pruebas

- Conversación multi-turno

- Historial de análisis

• Skills Directory** (`skills/`):

```
log-ingestion.py ■■ fetch_logs(source) ■■ Lee logs desde archivo ■■ Valida formato ■■ Limpia caracteres especiales log_pattern_matcher.py ■■ run_log_pattern_matcher(logs) ■■ Identifica patrones ■■ Cuenta ocurrencias ■■ Retorna estadísticas log_correlation_engine.py ■■ run_log_correlation_engine(events) ■■ Correlaciona eventos ■■ Identifica causas raíz ■■ Genera timeline anomaly_detector.py ■■ run_anomaly_detector(logs) ■■ Detecta anomalías ■■ Calcula probabilidades ■■ Genera alertas log_report_generator.py ■■ run_log_report_generator(analysis) ■■ Estructura datos ■■ Genera reportes ■■ Formatea salida servicenow_token_generator.py ■■ run_servicenow_token_generator() ■■ Obtiene token OAuth ■■ Maneja expiración ■■ Refresca automáticamente servicenow_incident_creator.py ■■ run_servicenow_incident_creator(incident) ■■ Crea tickets ■■ Vincula con CMDB ■■ Retorna ticket ID
```

■ FLUJO DE DATOS

Flujo 1: Análisis Completo de Logs (15-30 seg)

```
1. CLIENTE ■■ POST /api/analyze ■■ logs: "texto de logs" ■■ limit: 100 2. MULESOFT GATEWAY (8081) ■■ Recibe request ■■ Valida JSON ■■ DataWeave Transform ■■ Forward a Agent (5000) 3. PYTHON AGENT (5000) ■■ /api/analyze ■■ step: json_parsing ■■ step: log_ingestion 4. SKILL: Log Ingestion ■■ Parse log lines ■■ Validate format ■■ Clean text ■■ step: pattern_matching 5. SKILL: Pattern Matcher ■■ Identify patterns ■■ Count events ■■ Detect errors ■■ step: ollama_call 6. OLLAMA LLM (11434) ■■ Receive prompt ■■ Process with LLM ■■ Generate diagnosis ■■ Return analysis 7. SKILL: Report Generator ■■ Structure output ■■ Format response ■■ Add timestamps 8. PYTHON AGENT ■■ Cache response ■■ step: response_formatting ■■ Return to Mulesoft 9. MULESOFT GATEWAY ■■ Transform response ■■ Add metadata ■■ Return to Client 10. CLIENTE ■■ Recibe análisis completo ■■ annotated_events ■■ pattern_summary ■■ diagnostic_report ■■ timestamp
```

Flujo 2: Análisis Rápido sin IA (< 2 seg)

```
1. CLIENTE ■■ GET /api/patterns 2. MULESOFT GATEWAY ■■ Forward a Agent (5000) 3. PYTHON AGENT ■■ /api/patterns ■■ Retrieve default logs ■■ Skip LLM call 4. SKILL: Pattern Matcher ■■ Fast pattern detection ■■ Return cached results ■■ Minimal processing 5. CLIENTE ■■ Recibe patrones inmediatamente
```

Flujo 3: Creación de Incidentes (2-5 seg)

```
1. CLIENTE ■■ POST /api/incidents ■■ incident_data 2. MULESOFT GATEWAY ■■ Forward a
Agent (5000) 3. PYTHON AGENT ■■ /api/incidents ■■ Call ServiceNow skill 4. SKILL:
ServiceNow Token ■■ Generate OAuth token ■■ Validate credentials 5. SKILL: ServiceNow
Incident Creator ■■ Format incident payload ■■ Create ticket via REST ■■ Return ticket
ID 6. CLIENTE ■■ Recibe ticket ID ■■ Puede trackear en ServiceNow
```

■ REQUISITOS TÉCNICOS

Hardware Recomendado

```
MÍNIMO (para desarrollo): ■■ CPU: 4 cores @ 2.0 GHz ■■ RAM: 8 GB ■■ Disk: 30 GB SSD ■■
Conexión: 100 Mbps PRODUCCIÓN: ■■ CPU: 8+ cores @ 2.4+ GHz ■■ RAM: 32 GB ■■ Disk: 100+
GB SSD ■■ Conexión: 1 Gbps
```

Software Requerido

Dependencias Python

```
requests&amp;gt;=2.31.0 # HTTP client python-dotenv&amp;gt;=1.0.1 # Environment management
python-dateutil&amp;gt;=2.9.0 # Date parsing responses&amp;gt;=0.25.0 # API mocking (optional)
flask&amp;gt;=2.0.0 # Web framework
```

Dependencias Maven (Mule)

```
&amp;lt;dependency&amp;gt; &amp;lt;groupId&amp;gt;com.mulesoft.connectors&amp;lt;/groupId&amp;gt;
&amp;lt;artifactId&amp;gt;mule-http-connector&amp;lt;/artifactId&amp;gt;
&amp;lt;version&amp;gt;1.7.3&amp;lt;/version&amp;gt; &amp;lt;/dependency&amp;gt; &amp;lt;!-- Más dependencias en
pom.xml --&amp;gt;
```

Puertos Requeridos

```
8081 - Mulesoft Omni Gateway (HTTP REST) 5000 - Python Agent (HTTP REST) 11434 - OLLAMA
LLM (HTTP) 3306 - MySQL (si se usa integración)
```

Modelos OLLAMA Disponibles

```
llama2 (7B) - Recomendado, balance latencia/calidad phi (2.7B) - Ultraligero, mejor
latencia qwen (7B) - Alternativa, mejor español mistral (7B) - General purpose
neural-chat (7B) - Optimizado chat
```

■ INSTALACIÓN Y CONFIGURACIÓN

FASE 1: Preparación del Ambiente (30 min)

Paso 1.1: Verificar Requisitos

```
# Verificar Python python --version # Output: Python 3.9.x (mínimo) # Verificar Java
java -version # Output: openjdk 17+ o superior # Verificar Maven mvn --version # Output:
Apache Maven 3.9.x
```

Paso 1.2: Clonar/Verificar Repositorio

```
cd "C:\Users\dcamachoj\Desktop\Agente Trazas OLLAMA" # Verificar estructura ls # Debe
ver: log-analyzer-agent, log-analyzer-omni-gateway, README.md, etc.
```

Paso 1.3: Instalar OLLAMA

• Windows:**

```
# Opción 1: Descarga manual desde https://ollama.ai/download # Opción 2: Con Chocolatey
choco install ollama # Opción 3: Verificar si ya está instalado ollama --version
```

• Verificar instalación:**

```
# Terminal 1: Iniciar servicio ollama serve # Debe ver: Listening on 127.0.0.1:11434 #
Terminal 2: Verificar curl.exe http://localhost:11434/api/tags # Debe retornar JSON con
modelos
```

FASE 2: Configurar Python Agent (20 min)

Paso 2.1: Crear Entorno Virtual

```
cd log-analyzer-agent # Crear venv python -m venv venv # Activar venv
.\venv\Scripts\Activate.ps1 # Si da error de ejecución: Set-ExecutionPolicy
-ExecutionPolicy RemoteSigned -Scope CurrentUser # Luego repetir el Activate.ps1
```

• Verificación**: Terminal debe mostrar `(venv)` al inicio

Paso 2.2: Instalar Dependencias

```
# Asegurarse venv activado # (debe ver (venv) en terminal) # Instalar requerimientos pip
install -r requeriments.txt # Verificar instalación pip list # Debe ver: requests,
flask, python-dotenv, etc.
```

Paso 2.3: Configurar Variables de Entorno

```
# Crear archivo .env (basado en .env.example) Copy-Item .env.example -Destination .env
# Editar .env con valores reales notepad .env
```

• Contenido .env**:

```
# OLLAMA Configuration OLLAMA_API_URL=http://localhost:11434 OLLAMA_MODEL=llama2
OLLAMA_TIMEOUT=30 # Agent Configuration AGENT_PORT=5000 AGENT_DEBUG=false # ServiceNow
Integration (opcional) SERVICENOW_INSTANCE=dev12345 SERVICENOW_CLIENT_ID=your_client_id
SERVICENOW_CLIENT_SECRET=your_client_secret SERVICENOW_CREATE_INCIDENTS=false
```

FASE 3: Instalar y Ejecutar Python Agent (10 min)

Paso 3.1: Ejecutar Agent

```
# Asegurarse en log-analyzer-agent con venv activado # Iniciar agente python api.py #
Esperado output: # * Running on http://localhost:5000 # * Debug mode: off
```

Paso 3.2: Validar Agent

```
# Terminal nueva (sin cerrar agent) # Validar health curl.exe
http://localhost:5000/api/health # Esperado output: # {"status":"OK","service":"Log
Analyzer Agent","model":"llama2","ollama_url":"http://localhost:11434"}
```

FASE 4: Compilar y Ejecutar Mulesoft (30 min)

Paso 4.1: Compilar Gateway

```
cd log-analyzer-omni-gateway # Limpiar y compilar mvn clean package -DskipTests #
Esperado: # BUILD SUCCESS # Total time: X.XXs
```

Paso 4.2: Ejecutar Gateway

• Opción A: Desde terminal**

```
# En misma carpeta (log-analyzer-omni-gateway) mule -app
target/log-analyzer-omni-gateway-1.0.0-SNAPSHOT-mule-application.jar # Esperado: #
Started app 'log-analyzer-omni-gateway' # Listening on: 0.0.0.0:8081
```

• Opción B: Desde Anypoint Studio (GUI)**

```
1. Open Anypoint Studio 2. File → Open Project → log-analyzer-omni-gateway 3.
Right-click proyecto → Run As → Mule Application 4. Consola debe mostrar: "Started
app"
```

Paso 4.3: Validar Gateway

```
# Terminal nueva curl.exe http://localhost:8081/api/health # Esperado: # { # "status":
"OK", # "service": "Log Analyzer Gateway", # "agent_url": "http://localhost:5000" # }
```

FASE 5: Validar Integración Completa (15 min)

Paso 5.1: Test E2E

```
# Health check curl.exe http://localhost:8081/api/health # Analizar logs de ejemplo
$body = @{ logs = "2026-07-31T08:00:00Z [INFO] System startup`n2026-07-31T08:15:00Z
[WARNING] High memory" limit = 100 only_pattern_analysis = $false } | ConvertTo-Json
curl.exe -X POST ` -H "Content-Type: application/json" ` -d $body `
http://localhost:8081/api/analyze # Esperado: Análisis completo con diagnóstico
```

Paso 5.2: Usar Postman (Recomendado)

```
1. Importar: Log_Analyzer_API_V2.postman_collection.json 2. Variables → base_url:
http://localhost:8081 3. Ejecutar requests de ejemplo
```

■ GUÍA DE USO

Uso Básico: Analizar Logs

Método 1: URL + curl

```
curl -X POST \ -H "Content-Type: application/json" \ -d '{ "logs":
"2026-07-31T08:00:00Z [INFO] Startup\n2026-07-31T08:15:00Z [ERROR] Timeout", "limit":
100, "only_pattern_analysis": false }' \ http://localhost:8081/api/analyze
```

Método 2: Archivo local

```
# Leer contenido del archivo $logContent = Get-Content "C:\logs\mylog.txt" -Raw #
Enviar a API $body = @{ logs = $logContent limit = 100 only_pattern_analysis = $false }
| ConvertTo-Json curl.exe -X POST ` -H "Content-Type: application/json" ` -d $body `
http://localhost:8081/api/analyze
```

Método 3: Postman (GUI)

```
1. Request: POST http://localhost:8081/api/analyze 2. Headers: Content-Type:
application/json 3. Body (raw, JSON): { "logs": "... your logs ...", "limit": 100,
"only_pattern_analysis": false } 4. Click: Send
```

Uso Avanzado: Carga de Archivos

Opción 1: Form-Data Upload

```
curl -X POST \ -F "log_file=@mylog.txt" \ -F "limit=100" \ -F
"only_pattern_analysis=false" \ http://localhost:8081/api/analyze-file
```

Opción 2: Postman File Upload

```
1. Request: POST http://localhost:8081/api/analyze-file 2. Body → form-data 3. Key:
log_file, Value: Select File → mylog.txt 4. Key: limit, Value: 100 5. Key:
only_pattern_analysis, Value: false 6. Send
```

Análisis Rápido (Sin IA)

```
# Obtener patrones sin esperar LLM curl http://localhost:8081/api/patterns # Respuesta
instantánea (&amp;lt; 2 seg) # { # "patterns": [...], # "error_count": 5, # "warning_count":
```

```
3, # "info_count": 10 # }
```

Crear Incidentes (ServiceNow)

```
curl -X POST \ -H "Content-Type: application/json" \ -d '{ "title": "Database Timeout
Detected", "severity": "high", "description": "Multiple connection timeouts detected",
"logs_summary": "..." }' \ http://localhost:8081/api/incidents # Respuesta: # { #
"status": "success", # "ticket_id": "INC0012345", # "url":
"https://your-instance.service-now.com/nav_to.do?uri=incident.do?sys_id=..." # }
```

■ API REST - DOCUMENTACIÓN COMPLETA

Base URL

```
http://localhost:8081/api
```

Endpoints Disponibles

1. Health Check

• Endpoint**: `GET /api/health`

• Descripción**: Verificar estado de los servicios

• Respuesta exitosa (200)**:

```
{ "status": "OK", "service": "Log Analyzer Gateway", "agent_url":
"http://localhost:5000", "timestamp": "2026-09-14T10:30:00Z" }
```

• Ejemplo**:

```
curl http://localhost:8081/api/health
```

2. Analizar Logs (Completo)

• Endpoint**: `POST /api/analyze`

• Descripción**: Análisis completo con IA - Detección de patrones, anomalías y diagnóstico

• Headers requeridos**:

```
Content-Type: application/json
```

• Body (JSON)**:

```
{ "logs": "string", // (requerido) Texto de logs "limit": 100, // (opcional) Máx líneas
(default: 100) "only_pattern_analysis": false // (opcional) Sin IA (default: false) }
```

• Parámetros**:

• Respuesta exitosa (200)**:

```
{ "status": "success", "step": "response_formatting", "annotated_events": [ {
"timestamp": "2026-07-31T08:00:00Z", "level": "INFO", "message": "System startup
initiated", "event_type": "system_event", "severity": "low" }, { "timestamp":
"2026-07-31T08:25:00Z", "level": "ERROR", "message": "Database connection timeout",
"event_type": "database_error", "severity": "high", "related_events": 3 } ],
"pattern_summary": [ { "pattern": "Database connection timeout", "count": 3,
"percentage": 30, "severity": "HIGH", "trend": "increasing" }, { "pattern": "High
memory usage", "count": 1, "percentage": 10, "severity": "MEDIUM", "trend": "stable" }
], "diagnostic_report": "La solución presenta múltiples timeouts de conexión a la base
de datos (3 ocurrencias). Esto sugiere un problema de conectividad o saturación de la
piscina de conexiones. Recomendación: Revisar configuración de timeout y aumentar pool
size.", "execution_time_ms": 18500, "timestamp": "2026-09-14T10:30:45Z" }
```

• Respuesta error (400)**:

```
{ "status": "error", "message": "400 Bad Request: JSON parsing error", "hint":
"Verifique que Content-Type sea 'application/json' y JSON esté bien formado", "step":
"json_parsing" }
```

• Tiempos esperados**:

• Sin caché: 15-30 segundos

• Con caché: < 500 ms

• Ejemplo**:

```
curl -X POST \ -H "Content-Type: application/json" \ -d '{ "logs":
"2026-07-31T08:00:00Z [INFO] System startup\n2026-07-31T08:15:00Z [ERROR] Database
timeout", "limit": 100, "only_pattern_analysis": false }' \
http://localhost:8081/api/analyze
```

3. Análisis Rápido (Patrones)

• Endpoint**: `GET /api/patterns`

• Descripción**: Análisis rápido sin IA - Solo detección de patrones

• Parámetros Query**:

```
?limit=100&file=test_logs.txt
```

• Respuesta exitosa (200)**:

```
{ "status": "success", "patterns": [ { "pattern": "Database connection timeout",
"count": 3, "level": "ERROR", "first_occurrence": "2026-07-31T08:25:00Z",
"last_occurrence": "2026-07-31T08:25:02Z" }, { "pattern": "High memory usage detected",
"count": 1, "level": "WARNING" } ], "summary": { "total_events": 10, "error_count": 3,
"warning_count": 1, "info_count": 6, "critical_count": 2 }, "execution_time_ms": 800,
```

```
"timestamp": "2026-09-14T10:31:00Z" }
```

• Tiempos esperados**: < 2 segundos

• Ejemplo**:

```
curl "http://localhost:8081/api/patterns?limit=100"
```

4. Crear Incidente (ServiceNow)

• Endpoint**: `POST /api/incidents`

• Descripción**: Crear automáticamente un ticket en ServiceNow

• Headers requeridos**:

```
Content-Type: application/json
```

• Body (JSON)**:

```
{ "title": "string", // (requerido) Título del incidente "severity": "low|medium|high",
// (requerido) Severidad "description": "string", // (requerido) Descripción
"logs_summary": "string" // (opcional) Resumen del análisis }
```

• Respuesta exitosa (200)**:

```
{ "status": "success", "ticket_id": "INC0012345", "ticket_url":
"https://your-instance.service-now.com/nav_to.do?uri=incident.do?sys_id=abc123def456",
"created_at": "2026-09-14T10:32:00Z" }
```

• Tiempos esperados**: 2-5 segundos

• Ejemplo**:

```
curl -X POST \ -H "Content-Type: application/json" \ -d '{ "title": "Database Connection
Timeout", "severity": "high", "description": "Multiple connection timeouts detected",
"logs_summary": "3 ocurrencias de timeout en 30 segundos" }' \
http://localhost:8081/api/incidents
```

5. Carga de Archivo

• Endpoint**: `POST /api/analyze-file`

• Descripción**: Enviar archivo de log directamente

• Headers requeridos**:

```
multipart/form-data
```

• Body (Form-Data)**:

• Respuesta exitosa (200)**:

```
{ "status": "success", "file_name": "mylog.txt", "file_size": 5240, "lines_processed":
85, "...": "resto del análisis igual a /analyze" }
```

• Ejemplo (PowerShell)**:

```
$file = "C:\logs\mylog.txt" $url = "http://localhost:8081/api/analyze-file" $fileByte =
[System.IO.File]::ReadAllBytes($file) $fileName = Split-Path $file -Leaf $boundary =
[System.Guid]::NewGuid().ToString() $body = @" --$boundary Content-Disposition:
form-data; name="log_file"; filename="$fileName" Content-Type: text/plain $(Get-Content
$file -Raw) --$boundary Content-Disposition: form-data; name="limit" 100 --$boundary--
"@ $headers = @{ "Content-Type" = "multipart/form-data; boundary=$boundary" }
Invoke-WebRequest -Uri $url -Method Post -Body $body -Headers $headers
```

Códigos de Respuesta HTTP

Headers Recomendados

```
Content-Type: application/json; charset=utf-8 Accept: application/json User-Agent:
MyClient/1.0 X-Request-ID: unique-request-id (para tracking)
```

■ CASOS DE USO

Caso 1: Monitoreo de Logs en Tiempo Real

• Escenario**: Recibir logs de un servidor y detectar problemas automáticamente

• Proceso**:

1. Servicio Python escribe logs a `logs/production.log`

2. Script cron ejecuta análisis cada 5 minutos

3. Si encuentra errores, crea incidente en ServiceNow

• Implementación**:

```
# Script PowerShell: check_logs.ps1 while ($true) { # Leer logs nuevos $logs =
Get-Content "C:\logs\production.log" -Tail 100 # Enviar a análisis $body = @{ logs =
$logs | Out-String limit = 100 only_pattern_analysis = $false } | ConvertTo-Json
$response = curl.exe -X POST ` -H "Content-Type: application/json" ` -d $body `
http://localhost:8081/api/analyze | ConvertFrom-Json # Si hay errores críticos if
($response.pattern_summary.severity -eq "CRITICAL") { # Crear incidente curl.exe -X
POST ` -H "Content-Type: application/json" ` -d $response `
http://localhost:8081/api/incidents } Start-Sleep -Seconds 300 # Esperar 5 min }
```

Caso 2: Análisis Post-Mortem de Incidentes

• Escenario**: Después de un incident, analizar logs para encontrar causa raíz

• Proceso**:

1. Descargar logs del período afectado

2. Enviar a análisis completo

3. Generar reporte con diagnóstico

4. Presentar findings al equipo

• Implementación**:

```
# Exportar logs de Kubernetes (ejemplo) kubectl logs --all-containers=true \ -n
production \ --since=2h \ &gt; production_logs.txt # Analizar curl -X POST \ -H
"Content-Type: application/json" \ -d '{ "logs": '$(cat production_logs.txt | jq -R
-s)'", "only_pattern_analysis": false }' \ http://localhost:8081/api/analyze &gt;
analysis_report.json # Ver diagnóstico cat analysis_report.json | jq
'.diagnostic_report'
```

Caso 3: Integración con Sistema de Alertas

• Escenario**: Correlacionar logs de múltiples servidores

• Proceso**:

1. Recopilar logs de: App Server, DB Server, Cache Server

2. Enviar para correlación

3. Identificar causa común

4. Notificar al equipo de ops

• Implementación**:

```
import requests import json # Recopilar logs logs_app =
get_logs('http://app-server/logs') logs_db = get_logs('http://db-server/logs')
logs_cache = get_logs('http://cache-server/logs') # Combinar con contexto combined_logs
= f""" === APP SERVER === {logs_app} === DATABASE SERVER === {logs_db} === CACHE SERVER
=== {logs_cache} """ # Enviar para análisis response = requests.post(
'http://localhost:8081/api/analyze', json={ 'logs': combined_logs, 'limit': 500,
'only_pattern_analysis': False } ) # Procesar diagnóstico analysis = response.json()
print(f"Root Cause: {analysis['diagnostic_report']}") # Si es crítico, crear ticket if
'critical' in analysis['diagnostic_report'].lower(): requests.post(
'http://localhost:8081/api/incidents', json={ 'title': 'Critical Issue Detected',
'severity': 'high', 'description': analysis['diagnostic_report'] } )
```

Caso 4: Auditoría de Seguridad

• Escenario**: Detectar intentos de acceso no autorizados en logs

• Proceso**:

1. Exportar logs de autenticación

2. Detectar patrones de ataque

3. Generar reporte de seguridad

4. Alertar a security team

• Implementación**:

```
# Filtrar logs de autenticación grep -E "Authentication|Failed login|Unauthorized" \
/var/log/auth.log &gt; auth_filtered.log # Enviar para análisis curl -X POST \ -F
"log_file=@auth_filtered.log" \ -F "only_pattern_analysis=false" \
http://localhost:8081/api/analyze-file \ | jq '.diagnostic_report'
```

■ OPTIMIZACIÓN DE RENDIMIENTO

Técnicas de Optimización Aplicadas

1. Caching Inteligente

```
# Cache basado en hash SHA256 RESPONSE_CACHE = {} def
cached_ollama_call(prompt_content: str) -&gt; str: cache_key =
hashlib.sha256(prompt_content.encode()).hexdigest()[:16] if cache_key in
RESPONSE_CACHE: return RESPONSE_CACHE[cache_key] # Realizar llamada response =
call_ollama(prompt_content) RESPONSE_CACHE[cache_key] = response return response
```

• Beneficio**: Reduce latencia de 20-30s a < 500ms para análisis repetidos

2. Streaming de Respuestas

```
# LLM streaming activado def call_ollama(prompt): response = requests.post(
f'{OLLAMA_API_URL}/api/generate', json={ 'model': MODEL_NAME, 'prompt': prompt,
'stream': True # &lt;-- Activado }, stream=True ) # Procesar respuesta en chunks
```

• Beneficio**: Percepto más rápido de primeros resultados

3. Limpieza de Texto

```
def clean_text_response(text: str) -&gt; str: """Remover caracteres especiales que
ralentizan procesamiento""" # Convertir escapes text = text.replace('\\n', '\n') #
Remover caracteres de control text = ''.join(c for c in text if ord(c) &gt;= 32 or c in
'\n\r\t') # Normalizar espacios return text.strip()
```

• Beneficio**: Reduce tamaño de payload 15-20%

4. Pool de Conexiones

```
# Reutilizar conexiones HTTP session = requests.Session() adapter = HTTPAdapter(
pool_connections=10, pool_maxsize=20, max_retries=3 ) session.mount('http://', adapter)
```

• Beneficio**: Reduce latencia de conexión 30-40%

Selección de Modelo

```
Latencia vs Calidad: RÁPIDO (&lt; 10 seg): ■■ phi (2.7B) ■■ Ultraligero, bueno para patrones ■■ Latencia: 3-8 segundos ■■ Mejor para detección rápida BALANCEADO (15-20 seg): ■■ llama2 (7B) - RECOMENDADO ■■ Balance óptimo latencia/calidad ■■ Latencia: 12-20 segundos ■■ Mejor para análisis general PRECISO (25-35 seg): ■■ qwen (7B) ■■ Mejor comprensión español ■■ Latencia: 20-35 segundos ■■ Mejor para reportes detallados
```

Configuración Recomendada

```
# Para máxima velocidad OLLAMA_MODEL=phi CACHE_ENABLED=true STREAMING_ENABLED=true BATCH_SIZE=50 # Para máxima calidad OLLAMA_MODEL=qwen CACHE_ENABLED=true STREAMING_ENABLED=false BATCH_SIZE=25
```

■ TROUBLESHOOTING

Problema 1: Connection Refused (8081)

• Error**:

```
curl: (7) Failed to connect to localhost port 8081: Connection refused
```

• Causa Probable**: Mulesoft no está corriendo

• Solución**:

```
# 1. Verificar si está corriendo Get-Process | grep mule # 2. Si no está, iniciar cd log-analyzer-omni-gateway mule -app target/log-analyzer-omni-gateway-*.jar # 3. Esperar 30-40 segundos a que inicie # 4. Validar curl http://localhost:8081/api/health
```

Problema 2: Connection Refused (5000)

• Error**:

```
curl: (7) Failed to connect to localhost port 5000: Connection refused
```

• Causa Probable**: Python agent no está corriendo

• Solución**:

```
# 1. Ir a carpeta del agente cd log-analyzer-agent # 2. Activar venv .\venv\Scripts\Activate.ps1 # 3. Iniciar python api.py # 4. Debe ver: # * Running on http://localhost:5000 # * Debug mode: off
```

Problema 3: OLLAMA Unavailable (11434)

• Error**:

```
{"status": "error", "message": "Failed to connect to OLLAMA at http://localhost:11434"}
```

• Causa Probable**: OLLAMA no está corriendo o no tiene modelo

• Solución**:

```
# 1. Iniciar OLLAMA ollama serve # 2. En terminal nueva, descargar modelo ollama pull
llama2 # 3. Verificar curl http://localhost:11434/api/tags # 4. Debe retornar modelos
disponibles
```

Problema 4: JSON Parsing Error (400)

• Error**:

```
{"status": "error", "message": "400 Bad Request: JSON parsing error"}
```

• Causa Probable**: Content-Type incorrecto o JSON malformado

• Solución**:

```
# 1. Verificar Content-Type curl -v http://localhost:8081/api/analyze # Debe incluir:
Content-Type: application/json # 2. Validar JSON $json = '{"logs":"test","limit":100}'
$json | ConvertFrom-Json # Si da error, JSON es inválido # 3. Ejemplo correcto: $body =
@{ logs = "test logs" limit = 100 only_pattern_analysis = $false } | ConvertTo-Json
curl.exe -X POST ` -H "Content-Type: application/json" ` -d $body `
http://localhost:8081/api/analyze
```

Problema 5: Timeout (30 segundos)

• Error**:

```
Connection timeout after 30000ms
```

• Causa Probable**: OLLAMA tardando demasiado o problema de performance

• Solución**:

```
# 1. Verificar CPU/RAM Get-Process python | Format-Table CPU, Memory, ProcessName # 2.
Cambiar a modelo más rápido # Editar .env: OLLAMA_MODEL=phi # 3. Reiniciar agent #
Ctrl+C en terminal, luego python api.py # 4. Aumentar timeout en Mulesoft #
log-analyzer-omni-gateway/src/main/mule/global-configs.xml # &lt;http:request-config
timeout="60000"/&gt; (60 segundos)
```

Problema 6: Out of Memory

• Error**:

```
FATAL: java.lang.OutOfMemoryError: Java heap space
```

• Causa Probable**: Dataset muy grande o fuga de memoria

• Solución**:

```
# 1. Aumentar heap de Mulesoft $env:MULE_OPTS = "-Xmx2g -Xms1g" # 2. Aumentar heap de Python # En agent.py, al inicio: import sys sys.setrecursionlimit(10000) # 3. Procesar logs en chunks # Limitar --limit a 100 líneas máximo # 4. Reiniciar servicios
```

Problema 7: ServiceNow Authentication Failed

• Error**:

```
{"status": "error", "message": "ServiceNow authentication failed"}
```

• Causa Probable**: Credenciales incorrectas o expiradas

• Solución**:

```
# 1. Verificar credenciales en .env notepad .env # 2. Obtener credenciales correctas # En ServiceNow: # - Admin → System OAuth → Application Registry # - Crear nueva app o verificar credentials # 3. Actualizar .env # SERVICENOW_INSTANCE=dev12345 # SERVICENOW_CLIENT_ID=your_id # SERVICENOW_CLIENT_SECRET=your_secret # 4. Reiniciar agent # Ctrl+C, luego python api.py # 5. Probar curl.exe -X POST ` -H "Content-Type: application/json" ` -d '{"title":"Test","severity":"low","description":"Test"}' ` http://localhost:8081/api/incidents
```

■■ ROADMAP FUTURO

Q4 2026 - Versión 1.1

• ■ Soporte para múltiples idiomas (no solo español)

• ■ Dashboard web de monitoreo en tiempo real

• ■ Exportación de reportes a PDF/Excel

• ■ Integración con Splunk/ELK Stack

• ■ API Webhooks para notificaciones

Q1 2027 - Versión 1.2

• ■ Machine Learning para predicción de fallos

- ■ Análisis de tendencias (trending analysis)
- ■ Integración con Jira para ticketing
- ■ Support para logs en tiempo real (streaming)
- ■ Benchmark comparativo de patrones

Q2 2027 - Versión 2.0

- ■ LLM Multi-modal (soportar imágenes, métricas)
- ■ Análisis de causalidad (Root Cause Analysis)
- ■ Distributed processing (análisis paralelo)
- ■ Custom models training
- ■ GraphQL API

■ SOPORTE Y CONTACTO

Reportar Issues

```
Email: support@example.com Slack: #log-analyzer-agent JIRA: Log-Analyzer project
```

Documentación Complementaria

- README.md - Guía rápida
- INTEGRATION_GUIDE.md - Integración detallada
- BACKEND_API_IMPLEMENTATION.md - Especificación API
- API_AVAILABLE_LOGS_ENDPOINT.md - Endpoints específicos

Recursos Externos

- OLLAMA Documentation: https://ollama.ai/docs
- Mulesoft Documentation: https://docs.mulesoft.com
- ServiceNow API: https://developer.servicenow.com/
- Documento generado**: Septiembre 2026
- Versión**: 1.0.0
- Licencia**: Propietaria

• Última revisión**: 2026-09-14

• Para preguntas o sugerencias sobre este documento, contacte al equipo de desarrollo.*

Documento generado automáticamente
Fecha: 14 de September de 2026
Sistema: Agente de Análisis de Logs con OLLAMA
Licencia: Propietaria

Para preguntas o sugerencias sobre este documento, contacte al equipo de desarrollo.